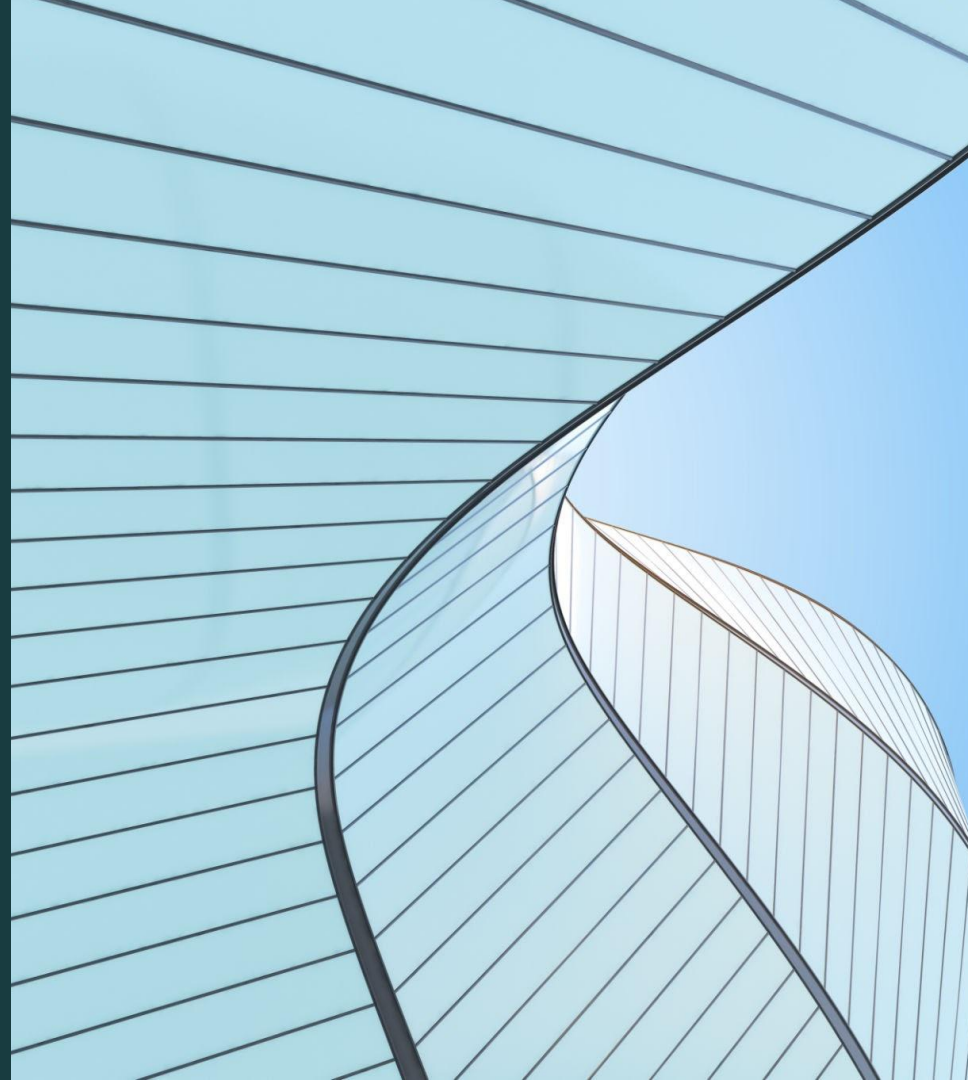


Skin Cancer Detection Milestone 1

By : Samuel Borges, Elif Dila Bayrak,
Neva Ildem, Sophia Zubieta



Progress Matrix Milestone 1

Task	Completion %	Elif	Neva	Samuel	Sophia	To do
Compare and Select Technical Tools	100%	25%	25%	25%	25%	none
“Hello World” Demos	100%	25%	25%	25%	25%	none
Resolve Technical Challenges	100%	25%	25%	25%	25%	none
Compare and Select Collaboration Tools	100%	25%	25%	25%	25%	none
Requirements Document	100%	25 %	25%	25%	25%	none
Design Document	100%	25%	25%	25%	25%	none
Test Plan	100%	25%	25%	25%	25%	none

Technical Tools Selection

Image Preprocessing : Python & OpenCV

Machine Learning Frameworks : TensorFlow/Keras

Evaluation Metrics: Scikit-Learn

Application Interface: Flutter, Android Studio, Dart, SQLite

Collaboration & Version Control : GitHub, GitHub Pages, Google Colab, AI Panther

Functional Requirements

- Image Input
 - Users may upload an image via the device's camera or gallery.
- Image Preprocessing
 - Process the image to match the format and requirements of training ISIC datasets.
- Machine Learning Model Training
 - Train CNN models using skin-lesion datasets.
- Machine Learning Model Evaluation
 - Evaluate the trained models using metrics such as accuracy, precision, and recall.
- Skin Lesion Prediction
 - Provide predictions of skin lesions (Benign or Malignant).
- Prediction Confidence Visualization
 - Provide a percentage of how certain the model is regarding prediction accuracy.
- Training Data Comparison
 - Compares images with thousands of images from the trained datasets.
- Safety Information and Disclaimers
 - Informs users that results are not a medical diagnosis to ensure safety.
- History and Lesion Tracking
 - Tracking the progress of the lesion through multiple image sessions.
- Data Management
 - Store and organize the user's account information and images, and implement security measures.
- Application Interface
 - Provide a simple application interface for easy navigation.

Non-functional Requirements

- Performance
 - Preprocess the image and provide the prediction in a short amount of time
- Scalability
 - Supports additional users, images and different features
- Accuracy
 - Model accurately identifies unseen images
- Reliability
 - Consistently produce results without crashing or handling errors
- Usability
 - Simple interface and intuitive for users with no technical experience
- Security
 - Protect data from unauthorized access
- Maintainability
 - Reliable code and structures to ensure easy modification of code if needed

Design & Test Documentation

- **Software Design Document**

- Defined Android client-server architecture
- Organized system into major software components
- Designed image preprocessing and ML inference workflow
- Defined database, image storage, and analysis history
- Established HTTPS communication and security approach
- Created Requirements Traceability Matrix

- **Software Test Plan**

- Created test cases for FR 1-11 and NFR 1-7
- Created tests for functional and non-functional requirements
- Defined Input/Action → Expected Result
- Included both normal and unusual error scenarios
- Covers image input, preprocessing, ML functions, prediction, history, data management, and interface
- Includes performance, accuracy, reliability, usability, security, and maintainability testing

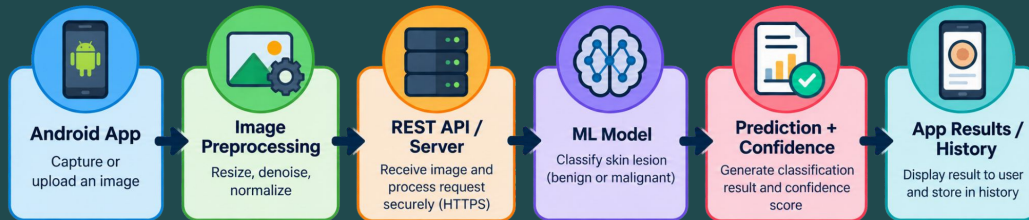
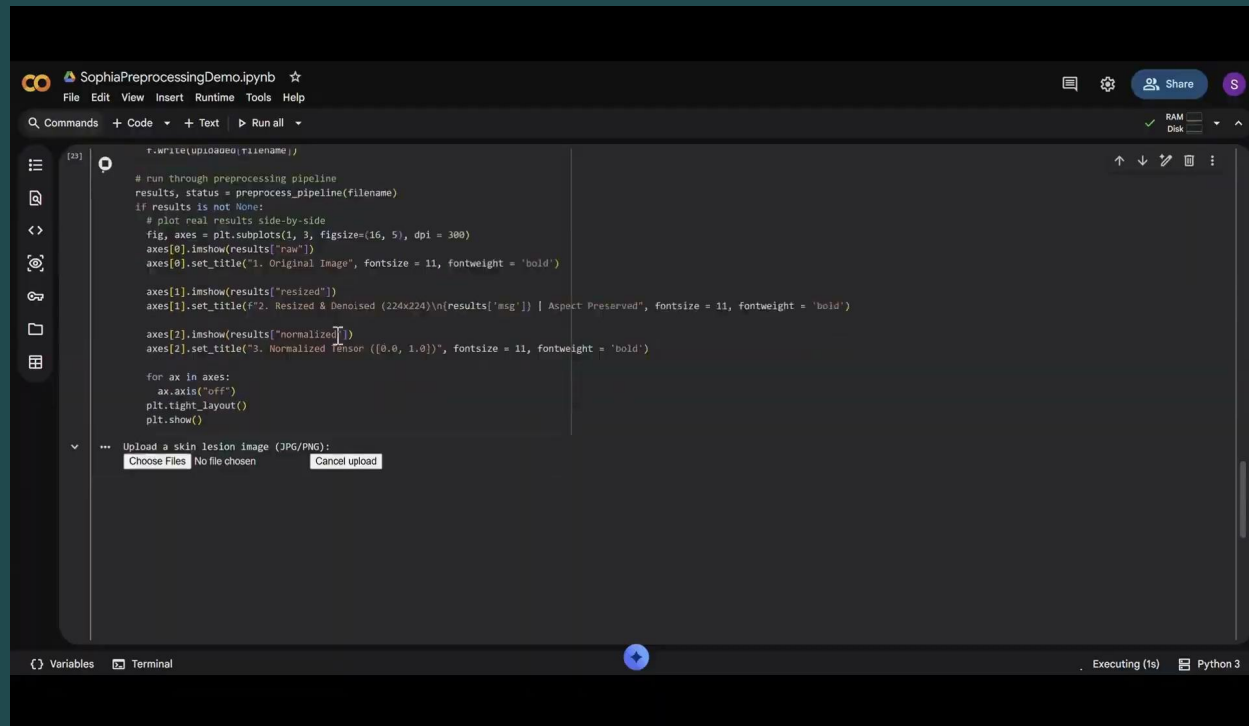


Image Preprocessing Pipeline Demo

- Simple hello world demo
 - user able to upload image of their choice to get preprocessed
 - resizes image without losing aspect ratio
 - denoises image without losing detailing
 - Normalizes image for deep learning model use
 - Shows output for each step of preprocessing



Preliminary Machine Learning Pipeline

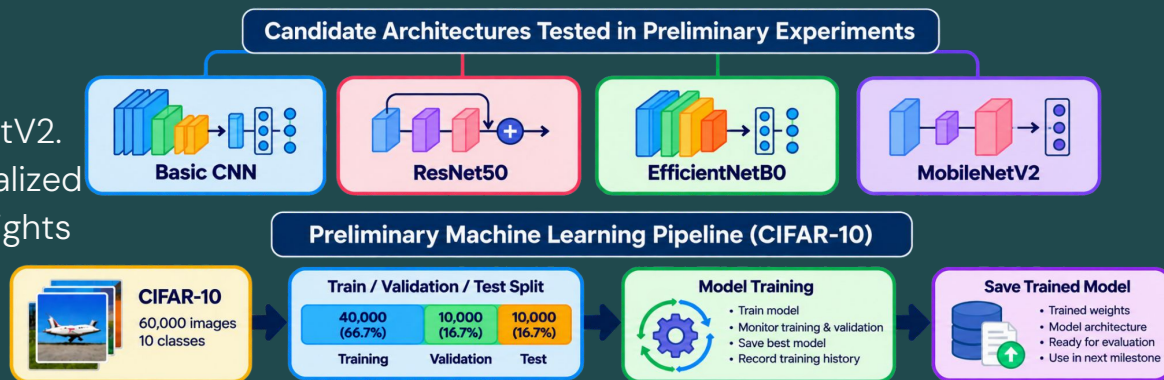
- Pipeline:

- CIFAR-10 → Train/Validation/Test Split → Model Training → Save Model

```
Epoch 8/10
1250/1250 ————— 6s 5ms/step - accuracy: 0.7438 - loss: 0.7311 - val_accuracy: 0.6884 - val_lo
ss: 0.9378
Epoch 9/10
1250/1250 ————— 5s 4ms/step - accuracy: 0.7615 - loss: 0.6842 - val_accuracy: 0.6698 - val_lo
ss: 0.9953
Epoch 10/10
1250/1250 ————— 6s 5ms/step - accuracy: 0.7748 - loss: 0.6469 - val_accuracy: 0.6678 - val_lo
ss: 1.0320
```

- Preliminary ML Pipeline Demo:

- Implemented a complete ML training pipeline using TensorFlow/Keras.
- Tested Basic CNN, ResNet50, EfficientNetB0, and MobileNetV2.
- Transfer learning models initialized with ImageNet pretrained weights
- CIFAR-10 used to verify the pipeline before skin-lesion training
- Models successfully trained and saved for evaluation



Model Evaluation Demo

For Milestone 1, we evaluated different machine learning models using CIFAR-10. The goal was to test our training and evaluation process before using the skin lesion dataset.

Preliminary Model Results

- Basic CNN: **66.08%**
- ResNet50: **37.50%**
- EfficientNetB0: **13.60%**
- MobileNetV2: **9.67%**

```
# Test the trained CNN using the test images
test_loss, test_accuracy = model.evaluate(x_test, y_test)

# Show the final results
print("Test Loss:", test_loss)
print("Test Accuracy:", test_accuracy)

313/313 ————— 1s 3ms/step - accuracy: 0.6608 - loss: 1.0306
Test Loss: 1.030598759651184
Test Accuracy: 0.6607999801635742
```

```
# Testing the trained ResNet50 model using the untouched CIFAR-10 test images.
resnet50_test_loss, resnet50_test_accuracy = resnet50_model.evaluate(
    x_test,
    y_test
)

# Displaying the final ResNet50 test results.
print("ResNet50 Test Loss:", resnet50_test_loss)
print("ResNet50 Test Accuracy:", resnet50_test_accuracy)

313/313 ————— 5s 15ms/step - accuracy: 0.3750 - loss: 1.8309
ResNet50 Test Loss: 1.8308974504470825
ResNet50 Test Accuracy: 0.375
```

```
# Test the trained EfficientNetB0 model using the test images.
efficientnet_test_loss, efficientnet_test_accuracy = best_efficientnet_model.evaluate(
    x_test,
    y_test
)

# Show the final results.
print("EfficientNetB0 Test Loss:", efficientnet_test_loss)
print("EfficientNetB0 Test Accuracy:", efficientnet_test_accuracy)

313/313 ————— 13s 23ms/step - accuracy: 0.1360 - loss: 2.3345
EfficientNetB0 Test Loss: 2.3345117568969727
EfficientNetB0 Test Accuracy: 0.13600000739097595
```

```
MobileNetV2 Test Loss: 123.85963439941406
MobileNetV2 Test Accuracy: 0.09669999778270721
```

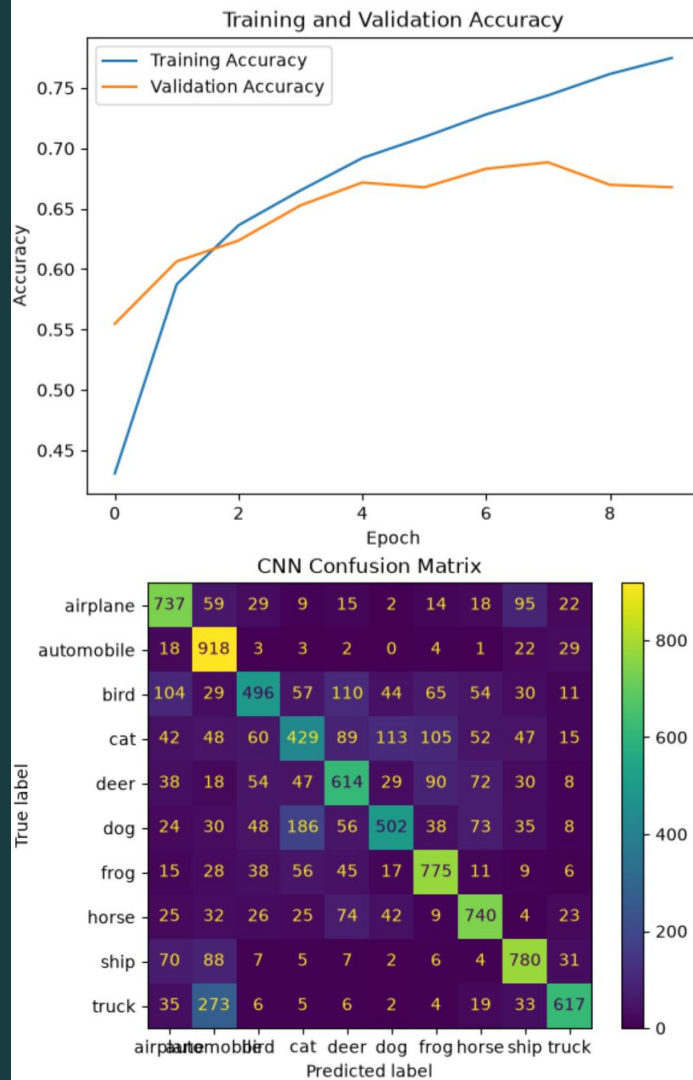
Basic CNN Evaluation

CIFAR-10 Performance Analysis

After testing the models, we evaluated the Basic CNN in more detail. We used different evaluation methods to understand how well the model performed and where it made mistakes.

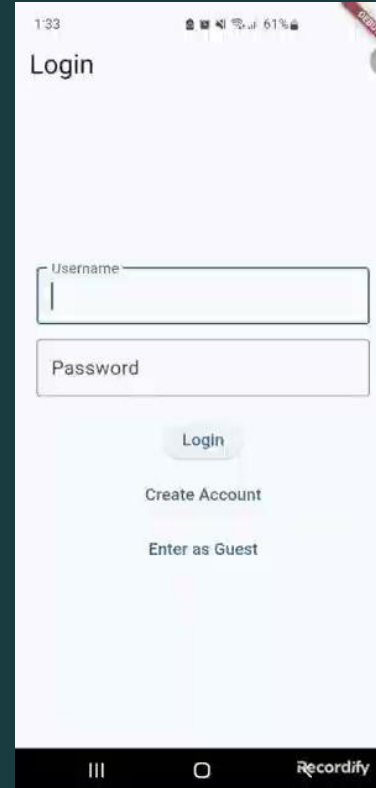
Evaluation Results

- Accuracy: **67.37%**
- Precision: **0.68**
- Recall: **0.67**
- F1 Score: **0.67**



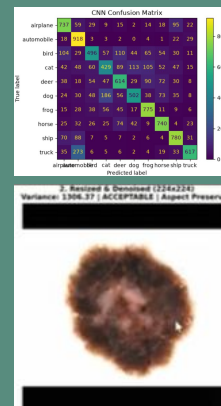
Application Interface Demo

- Flutter mobile application
- Dart used to develop screens, buttons and application logic
- SQLite used to store and retrieve user account information
- Android Studio used to provide Android development tools, deployment and testing on mobile devices
- Link to video demo:



Technical Challenges

- Machine Learning Model Selection
 - Reviewed recent skin lesion classification research and investigated basic CNN, ResNet50, EfficientNetB0, and MobileNetV2.
 - Final selection will be based on literature review findings and experiments using skin lesion data.
- Model Performance Evaluation
 - Accuracy alone is not sufficient.
 - Precision, recall/sensitivity, F1 score, confusion matrices, and false negatives will be evaluated.
- Image Quality & Preprocessing
 - Lesion images may vary in size, quality, noise, and appearance, requiring consistent preprocessing before inference.
- Mobile App & ML Integration
 - The Android application must securely send images to the server/API and receive classification and confidence results.



Milestone 2 Task Matrix

Task	Elif	Neva	Samuel	Sophia
Dataset & Preprocessing Pipeline Execution	-	Prepare dataset for model training.	-	Execute preprocessing pipeline.
CNN Model Training	Train and compare CNN models.	Train and compare CNN models.	-	-
Mobile App Frontend UI	-	-	Develop Android UI.	-
REST API Backend & Preprocessing Endpoint	-	-	-	Develop REST API & preprocessing endpoint.
Model Evaluation & Confusion Matrix	Evaluate models & metrics.	Evaluate models & metrics.	-	-
App to Server Image Upload	Support server connection and testing.	-	Implement image upload from app.	-

Thank you!